

SE RTU Solution 2023

Part A

Q1. What are requirement analysis tasks and principles?

Requirement Analysis Tasks

Requirement analysis involves:

- Requirement elicitation
- Requirement analysis & negotiation
- Requirement specification
- Requirement validation

Principles

1. Understand problem domain
2. Focus on user needs
3. Avoid design decisions
4. Ensure completeness and consistency

Q2. What is object modularization?

Object modularization is the process of dividing a software system into **independent, manageable objects or modules**, each responsible for a specific functionality, improving maintainability and reuse.

Q3. What is Unified Modeling Language (UML)?

UML is a **standard visual modeling language** used to specify, visualize, construct, and document **object-oriented software systems** using diagrams.

Q4. What is object-oriented analysis modeling?

Object-Oriented Analysis (OOA) modeling identifies system requirements by modeling real-world entities as **objects**, defining their attributes, behaviors, and relationships.

Q5. What is object-oriented design concept and method?

Object-Oriented Design (OOD) converts analysis models into a design model using classes, objects, and interfaces.

It uses methods like **encapsulation, inheritance, and polymorphism**.

SE RTU Solution 2023

Q6. What is behavior modeling?

Behavior modeling represents the **dynamic behavior** of a system by showing how objects interact and change states over time using sequence and state diagrams.

Q7. List out various activities that are encompassed by system design process under object-oriented design.

Activities include:

- Architectural design
- Class and object design
- Interface design
- Data design
- Component-level design

Q8. What is system-level project planning?

System-level project planning defines the overall project scope, resources, cost, schedule, and milestones required to successfully develop the complete software system.

Q9. What is finite state machine (FSM) model?

FSM model represents a system using **finite states and transitions**, where state changes occur based on inputs or events, commonly used in control systems.

Q10. What is sequence diagram in context of UML?

A **sequence diagram** is a UML interaction diagram that shows **time-ordered message exchanges** between objects to represent system behavior.

SE RTU Solution 2023

Part B

Q1. Description of Software Design Document. What is the significance of Design Document?

Software Design Document (SDD) –

A **Software Design Document (SDD)** is a **formal technical document** that describes **how a software system will be designed and built** to satisfy the requirements specified in the Software Requirement Specification (SRS). It acts as a **blueprint** for developers, testers, and stakeholders before actual coding begins.

The design document translates **what the system should do** (requirements) into **how the system will do it** (design). It provides a clear picture of the system's **architecture, modules, data structures, interfaces, and interactions**.

Contents of a Software Design Document

- **System Architecture** – Overall structure of the system
- **Module Design** – Description of individual modules and their functions
- **Data Design** – Data structures, databases, and file organization
- **Interface Design** – User interface and module interfaces
- **Control & Data Flow** – DFDs, CFDs, and interaction diagrams
- **Design Constraints** – Hardware, software, or performance limitations

Typical Software Design Document Diagram

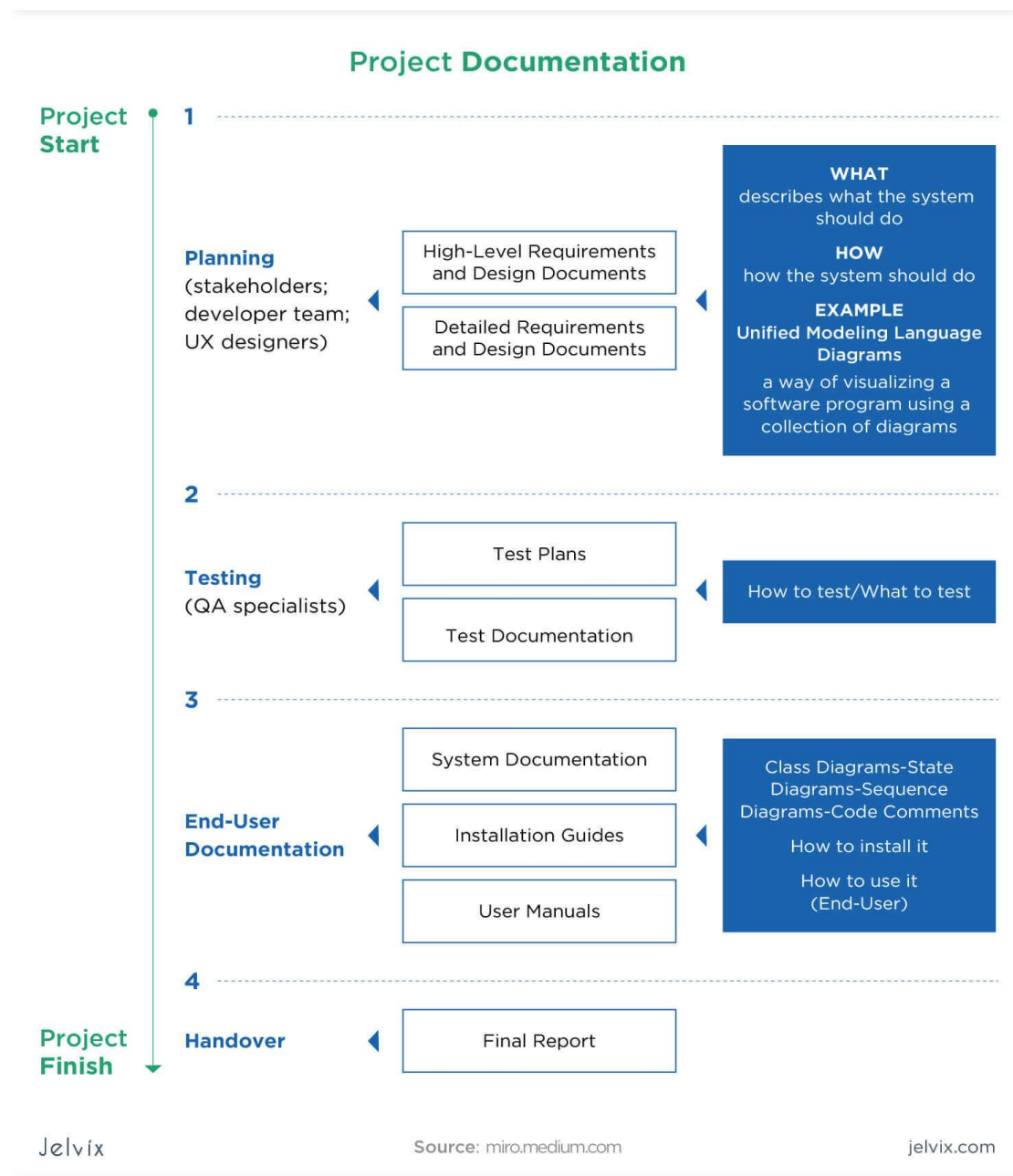


Significance of Software Design Document The design document plays a critical role in software development for the following reasons:

1. **Acts as a blueprint** Guides developers during coding and implementation.

SE RTU Solution 2023

2. **Improves understanding** Helps all stakeholders clearly understand system structure and behavior.
3. **Reduces development errors** Identifies design flaws early, saving cost and time.
4. **Supports modular development** Enables parallel development of different modules.
5. **Facilitates maintenance** Makes future enhancements and debugging easier.
6. **Ensures consistency** Maintains uniform design standards across the system.
7. **Assists testing and validation** Testers use the design document to create test cases.



SE RTU Solution 2023

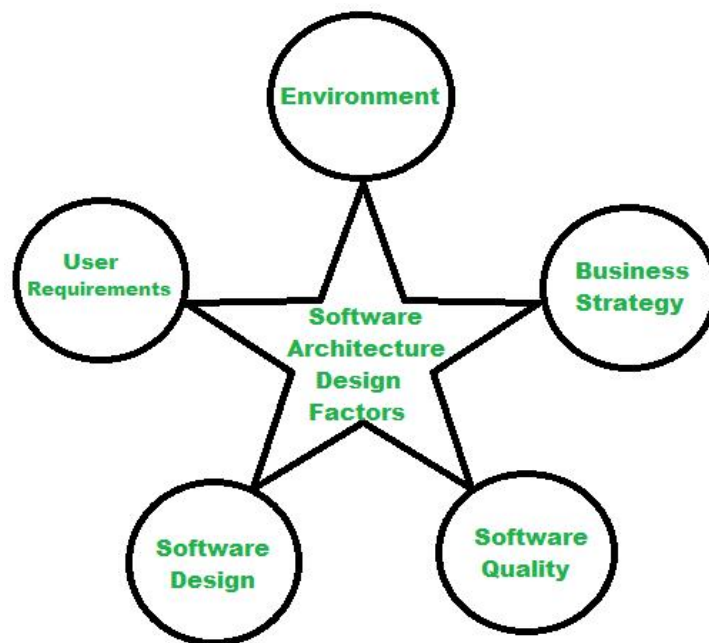
Q2. What is architectural and procedural software design? Explain.

Architectural Software Design

Architectural design defines the **overall structure** of a software system. It focuses on identifying **major components (modules/subsystems)**, their **responsibilities**, and the **relationships and interactions** among them. This design phase acts as a **high-level blueprint** of the system.

Key Points of Architectural Design

- Divides the system into **major components**
- Defines **data flow** and **control flow** between components
- Specifies **system structure**, layers, and subsystems
- Addresses **performance, scalability, security, and reliability**
- Helps in **risk reduction** and early validation of design decisions



Example

In a **College Management System**, major components may include:

1. Student Module
2. Faculty Module
3. Examination Module
4. Database Module

SE RTU Solution 2023

Procedural Software Design

Procedural design focuses on the **internal logic of individual modules**. It describes **how each module works**, including the **sequence of operations**, decision-making, and control structures.

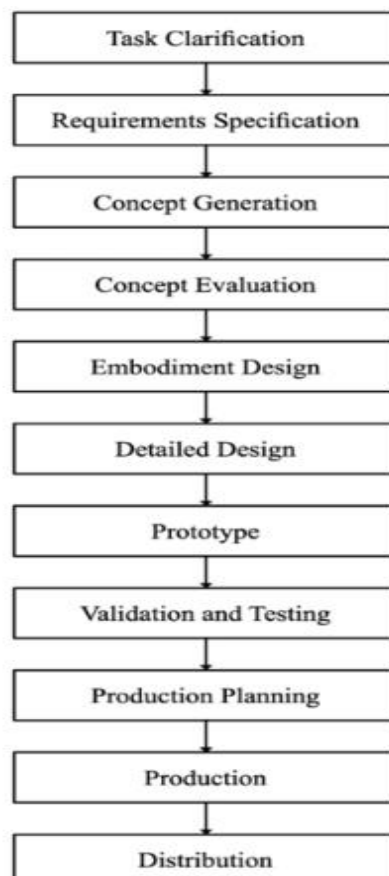
Key Points of Procedural Design

- Converts architectural components into **detailed algorithms**
- Uses **flowcharts, pseudocode, and control flow diagrams**
- Defines **step-by-step processing logic**
- Emphasizes **control structures** like loops and conditions
- Ensures correct and efficient implementation

Example

For a **Student Registration Module**, procedural design explains:

1. Input student details
2. Validate information
3. Store data in database
4. Generate confirmation



SE RTU Solution 2023

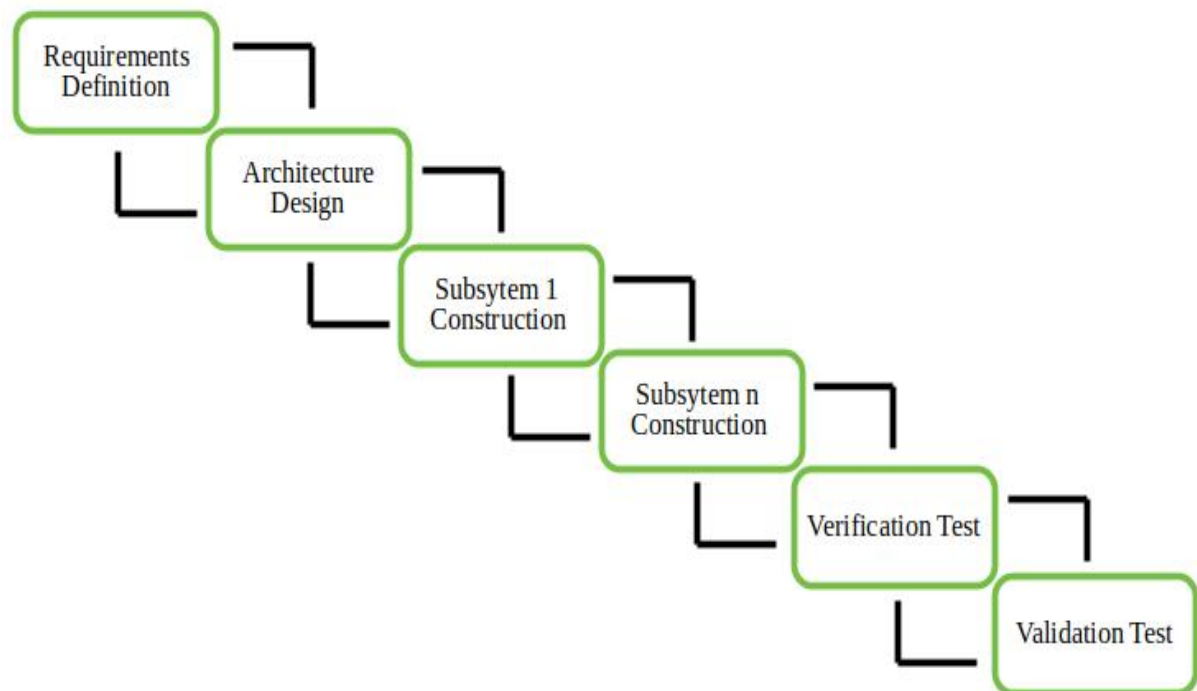
Q3. Explain Incremental Process Model. Justify its use for business software system but not for real-time system.

Incremental Model

The **Incremental Model** develops software in **small, manageable increments**, where each increment delivers **working functionality**.

Working

- System is divided into modules
- Each increment goes through SDLC phases
- Final system is a combination of all increments



Advantages

- Early delivery of software
- Easy to handle changes
- Reduced risk

Disadvantages

- Needs good planning
- Not suitable if system is highly interdependent

SE RTU Solution 2023

Applications

- Business applications
- Web applications

The Incremental Model is **well-suited for business software systems** such as banking systems, payroll systems, or inventory management because:

- Business requirements **change frequently**
- Early delivery of basic features is possible
- Continuous **user feedback** improves functionality
- Lower risk as system grows step by step
- Easy to add or modify features like reports, billing, or user roles
- Faster time-to-market for core business operations

Example: An **online shopping system** can first release product browsing, then add payment, and later delivery tracking.

Why NOT Suitable for Real-Time Systems

Real-time systems (e.g., aircraft control, medical monitoring) require **strict timing constraints and high reliability**, where the incremental model is less effective:

- Real-time systems need **complete system integration** from the start
- Partial or incomplete functionality can be **dangerous**
- Timing constraints cannot be tested properly in small increments
- High dependency between modules reduces flexibility
- Requires precise hardware-software synchronization
- Delayed integration may cause system failure

Example: An **aircraft control system** cannot be delivered in parts due to safety risks.

SE RTU Solution 2023

Q4. What is SDLC? Explain MIS-Oriented SDLC Model.

SDLC (Software Development Life Cycle)

SDLC (Software Development Life Cycle) is a systematic framework used to develop software in a disciplined manner, ensuring quality, cost control, and timely delivery. It defines the complete process from idea conception to software maintenance.

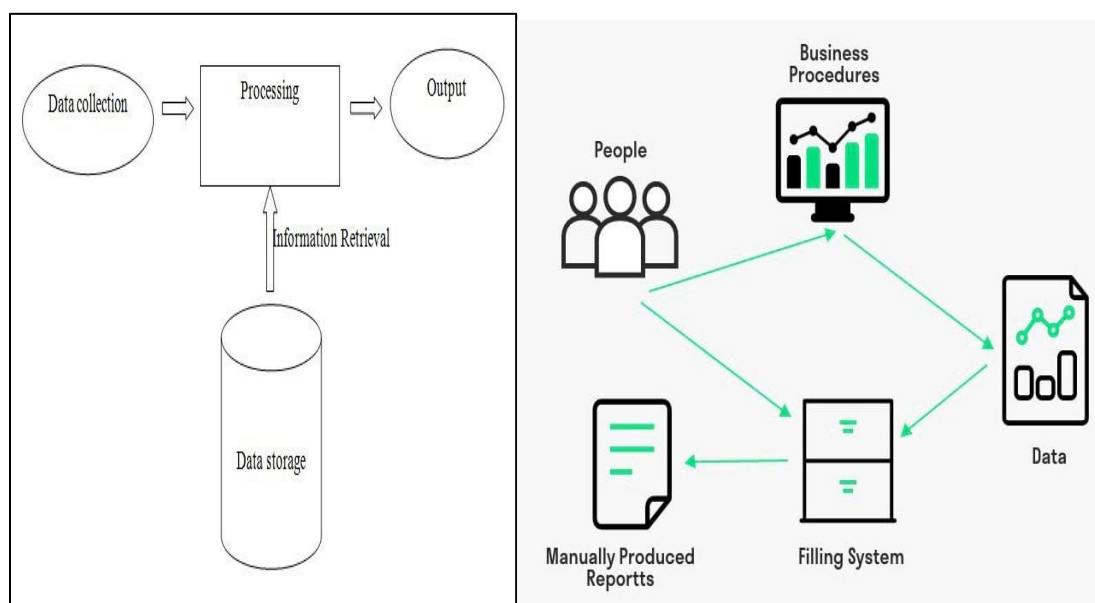
Phases of SDLC

1. Planning
2. Requirement Analysis
3. System Design
4. Coding
5. Testing
6. Implementation
7. Maintenance

MIS-Oriented SDLC Model

The **MIS-oriented SDLC model** is specifically designed for **Management Information Systems**, where the primary objective is to process data and generate meaningful information for decision-making. This model emphasizes structured documentation, data flow, and reporting accuracy.

MIS-SDLC Diagram



SE RTU Solution 2023

Planning → System Analysis → System Design → Development → Implementation → Operation & Maintenance

Features

- Data-centric
- Strong documentation
- Used in business organizations

Q5. What is Object-Oriented Design (OOD) concept? Explain.

Object-Oriented Design (OOD) is a software design approach that models a system as a collection of interacting **objects**, where each object represents a real-world entity. OOD helps in building **reusable, maintainable, and scalable** software systems.

Key OOD Concepts

- **Object** : An object is an instance of a class that contains **data (attributes)** and **behavior (methods)**.
- **Class** : A class is a blueprint or template used to create objects.
- **Encapsulation** : It binds data and methods together and hides internal details using access specifiers.
- **Abstraction** : It shows only essential features to the user and hides unnecessary implementation details.
- **Inheritance** : It allows one class to acquire properties and methods of another class, promoting code reuse.
- **Polymorphism** : It allows the same method name to behave differently based on the object or context.

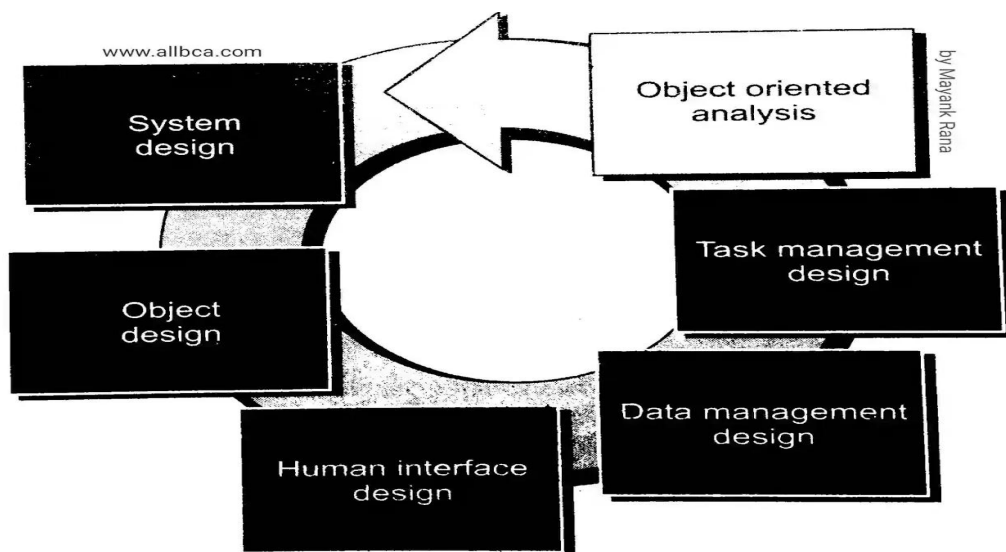


Fig. Process flow for object-oriented design.

SE RTU Solution 2023

Advantages of OOD

1. Improves code reusability
2. Enhances security and data hiding
3. Makes maintenance and debugging easier
4. Closely represents real-world systems

Q6. Explain class and object relationship in object-oriented analysis.

In **Object-Oriented Analysis (OOA)**, a system is modeled using **classes** and **objects** to represent real-world entities. Understanding the relationship between classes and objects is essential for designing clear, reusable, and maintainable software systems.

Class and Object

Class : A **class** is a blueprint or template that defines the **attributes (data members)** and **methods (functions)** common to all objects of that type.
Example: Student class with attributes like name, rollNo and method register().

Object : An **object** is a real instance of a class. It represents an actual entity and occupies memory.
Example: Student s1 = new Student("Amit", 101);

Relationship: A class can create **multiple objects**, but an object always belongs to **one class**.

Types of Relationships in Object-Oriented Analysis

1. Class–Object Relationship

Objects are **instances** of a class

Class defines structure; object represents reality

Class: Student
Objects: s1, s2, s3

2. Association

Represents a **logical relationship** between classes

Example: Student *enrolls in* Course

3. Aggregation (HAS-A, weak relationship)

SE RTU Solution 2023

One class contains another, but both can exist independently

Example: Department has Teachers

4. Composition (HAS-A, strong relationship)

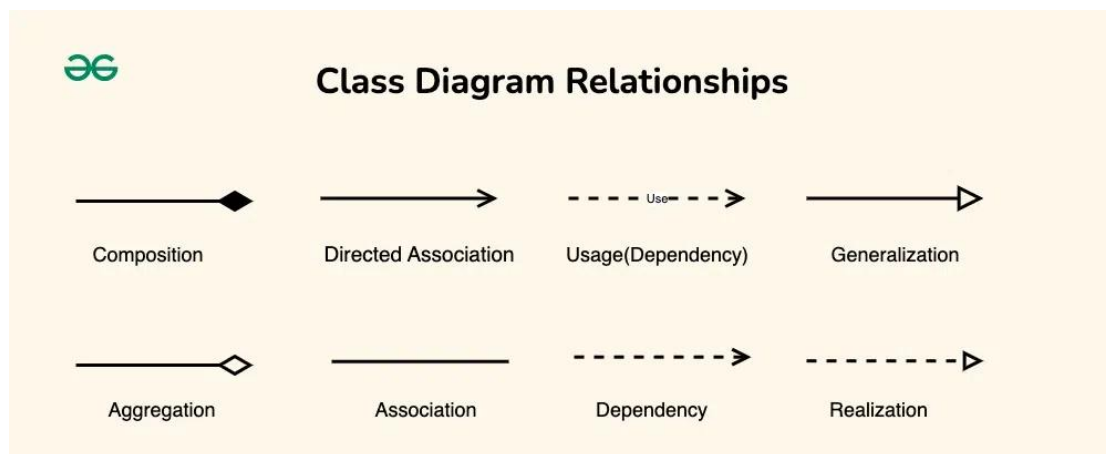
Child object cannot exist without parent

Example: House has Rooms

5. Inheritance (IS-A relationship)

One class inherits properties of another

Example: GraduateStudent inherits from Student



Q7.Explain COCOMO estimation model in software project management.

COCOMO (Constructive Cost Model) is an algorithmic software cost estimation model proposed by Barry W. Boehm. It is used in software project management to estimate the effort (person-months), development time, and cost required to develop a software project.

The estimation is mainly based on the size of the software, measured in **KLOC (Kilo Lines of Code)**.

COCOMO assumes that:

- Larger software requires more effort
- Project complexity affects development effort
- Different types of projects need different estimation formulas

SE RTU Solution 2023

Types of COCOMO Models

1 Basic COCOMO

Simple and quick estimation

Uses only **project size (KLOC)**

Formula:

$$\text{Effort} = a \times (\text{KLOC})^b$$

2 Intermediate COCOMO

Considers **cost drivers** (hardware, personnel, project attributes)

More accurate than Basic COCOMO

3 Detailed (Advanced) COCOMO

Most detailed and accurate

Estimates effort for **each phase of SDLC**

Used for large and complex systems

Modes of Development

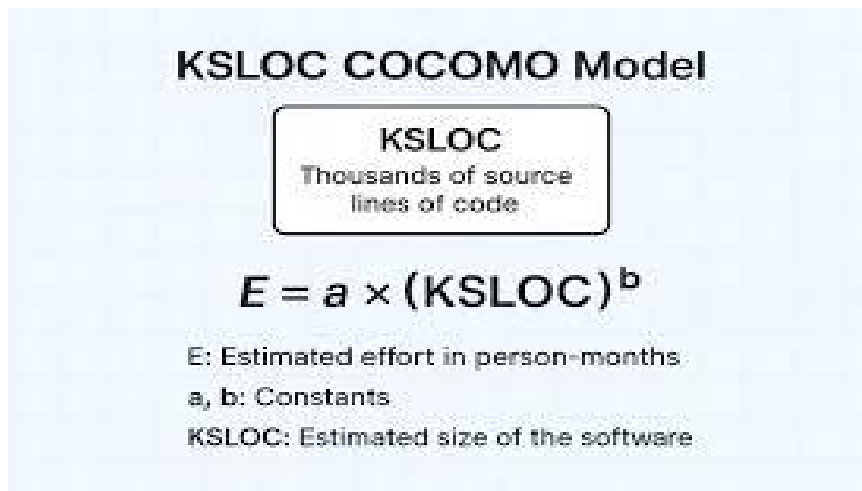
COCOMO Mode	Approximate LOC Range	Description
Organic	2,000 – 50,000 LOC (2–50 KLOC)	Small, simple projects with experienced teams and stable requirements
Semi-Detached	50,000 – 300,000 LOC (50–300 KLOC)	Medium-size projects with moderate complexity and mixed experience
Embedded	Above 300,000 LOC (>300 KLOC)	Large, complex projects with strict hardware, software, or real-time constraints

COCOMO classifies projects into three modes:

1. **Organic** – Small, simple projects with experienced teams
2. **Semi-Detached** – Medium-size projects
3. **Embedded** – Complex projects with strict constraints

SE RTU Solution 2023

COCOMO Estimation Diagram



Project Size (KLOC)



COCOMO Model



Effort (PM) → Time → Cost

Importance of COCOMO in Project Management

- Helps in **project planning and scheduling**
- Assists in **resource allocation**
- Reduces risk of cost and time overrun
- Provides a scientific basis for estimation

SE RTU Solution 2023

Part C

Q1. Discuss problems that occur while developing a system and suggest possible solutions

Developing a software or information system is a **complex process** involving users, developers, managers, tools, and technologies. During system development, various **technical, managerial, and human-related problems** arise that can affect **cost, schedule, quality, and user satisfaction**. Identifying these problems early and applying suitable solutions is essential for successful system development.

Major Problems in System Development and Their Solutions:

1. Unclear or Changing Requirements

Users may not clearly understand their needs, or requirements may change frequently during development.

Solution:

- Conduct detailed requirement analysis
- Use **prototyping** to clarify expectations
- Maintain continuous user involvement

2. Poor Planning and Estimation

Incorrect estimation of time, cost, and resources leads to delays and budget overruns.

Solution:

- Use estimation models like **COCOMO**
- Create realistic schedules and milestones
- Include contingency buffers

3. Communication Gap

Miscommunication between users, developers, and management causes misunderstandings and rework.

Solution:

- Conduct regular meetings and reviews
- Maintain proper documentation
- Use modeling tools such as **DFD and UML diagrams**

SE RTU Solution 2023

4. Lack of Skilled Manpower

Inexperienced developers may produce low-quality and error-prone systems.

Solution:

- Provide training programs
- Assign tasks based on expertise
- Encourage teamwork and mentoring

5. Poor System Design

Weak design leads to poor performance, low maintainability, and high coupling.

Solution:

- Follow modular design principles
- Ensure **high cohesion and low coupling**
- Review design documents thoroughly

6. Inadequate Testing

Insufficient testing results in undetected bugs and unreliable systems.

Solution:

- Perform unit, integration, system, and acceptance testing
- Use automated testing tools

Diagram: Problems and Solutions in System Development



SE RTU Solution 2023

Q2. Explain the System Development Plan in Detail

Introduction

A **System Development Plan (SDP)** is a formal document that describes **how a system will be developed, managed, and delivered** within specified time, cost, and quality constraints. It acts as a **roadmap** for the entire project and guides managers, developers, and stakeholders throughout the **System Development Life Cycle (SDLC)**.

The main objective of a system development plan is to ensure that the project is **well-organized, controlled, and completed successfully**.

Components of a System Development Plan

1. Project Overview

- Defines project goals and objectives
- Describes the system scope and expected outcomes
- Identifies stakeholders and users

2. Feasibility Analysis

- **Technical feasibility** – availability of technology
- **Economic feasibility** – cost–benefit analysis
- **Operational feasibility** – user acceptance
- **Schedule feasibility** – time constraints

3. Project Schedule and Milestones

- Divides the project into phases
- Defines deadlines and deliverables
- Uses tools like **Gantt charts and PERT charts**

4. Resource Planning

- Identifies manpower requirements
- Specifies hardware and software resources
- Assigns roles and responsibilities

5. Risk Management

- Identifies possible risks (technical, schedule, cost)
- Defines mitigation and contingency plans

SE RTU Solution 2023

6. Development Methodology

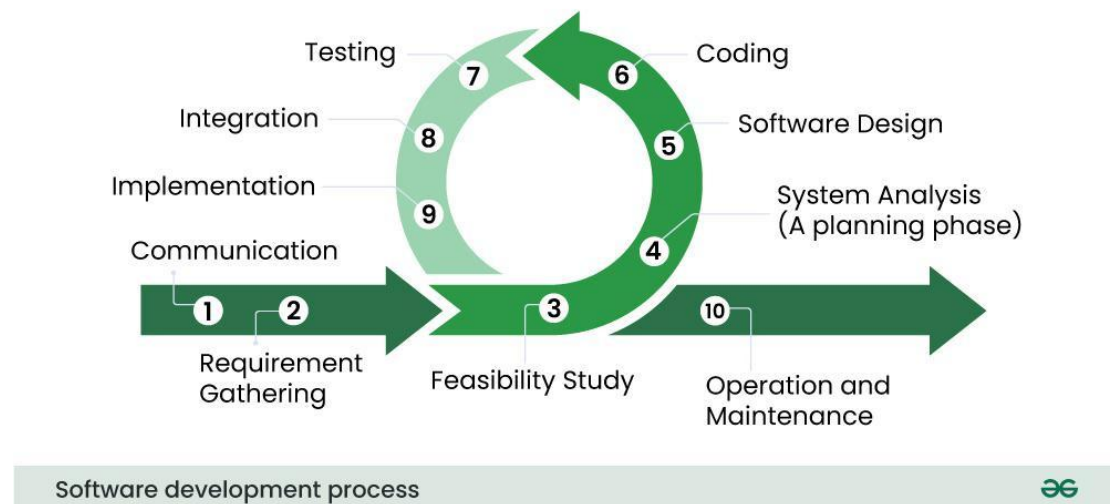
- Specifies SDLC model to be followed
- Examples: Waterfall, Incremental, Spiral, Agile

7. Quality Assurance and Testing Plan

- Defines testing strategies and standards
- Ensures reliability and correctness of the system

8. Documentation and Review Plan

- Lists documents like SRS, design document, test reports
- Specifies review and approval procedures



Q3. Explain Finite State Machine (FSM) Diagram in Detail

Introduction to Finite State Machine (FSM)

A **Finite State Machine (FSM)** is a **behavioral model** used in software engineering to represent a system that can exist in a **finite number of states** at any given time. The system changes from one state to another based on **inputs or events**. FSMs are widely used to model **control-oriented and event-driven systems** such as vending machines, traffic lights, communication protocols, and real-time systems.

FSM helps in understanding **how a system behaves**, rather than how data flows through it.

SE RTU Solution 2023

Basic Elements of FSM: An FSM consists of the following components:

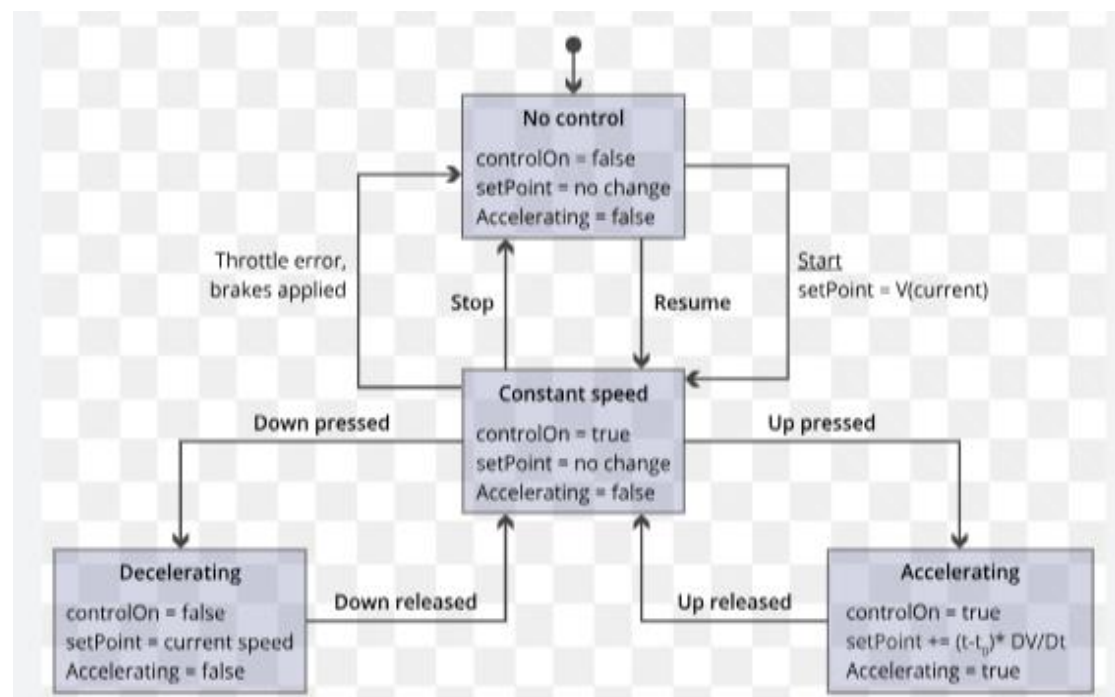
- 1) **States:** Represent different conditions or situations of the system.
Only one state is active at a time
- 2) **Events / Inputs:** External or internal triggers that cause state transitions
- 3) **Transitions:** Movement from one state to another due to an event
- 4) **Actions:** Activities performed during a transition or within a state
- 5) **Initial State:** The starting point of the system
- 6) **Final (Accepting) State:** Indicates completion of system operation

Working of Finite State Machine

1. The system starts in an **initial state**
2. When an **event occurs**, the system checks for a valid transition
3. If a valid transition exists, the system **moves to the next state**
4. An **action** may be executed during the transition
5. This process continues until a **final state** is reached

FSM ensures that the system behavior is **predictable, controlled, and easy to verify**.

Finite State Machine Diagram



SE RTU Solution 2023

Example Explanation (Vending Machine)

- 1) **Idle State** – Machine is waiting for user input
- 2) **Processing State** – Coin inserted and item selection is made
- 3) **Dispense State** – Item is delivered to the user

Each arrow represents a **transition**, and labels represent **events**.

Advantages of FSM

- Simple and easy to understand
- Clearly represents system behavior
- Useful for real-time and embedded systems
- Helps in validation and verification

Limitations of FSM

- Not suitable for very large and complex systems
- Number of states may increase rapidly
- Difficult to manage if behavior is highly dynamic

Q4. How to prepare a Software Requirement Specification document list possible users and uses of SRS in details with diagram.

A **Software Requirement Specification (SRS)** is a **formal document** that clearly describes **what a software system should do**. It acts as an agreement between the **client and the development team** and serves as the foundation for **design, development, testing, and maintenance**.

How to Prepare an SRS Document (Step-by-Step)

1. Requirement Gathering

- Collect requirements from users, customers, and stakeholders
- Use interviews, questionnaires, observations, and discussions

2. Requirement Analysis

- Analyze collected requirements for clarity and feasibility
- Remove ambiguities and conflicts
- Classify requirements into **functional** and **non-functional**

SE RTU Solution 2023

3. Structuring the SRS Document

A standard SRS (as per IEEE) includes:

- **Introduction** – purpose, scope, definitions
- **Overall Description** – system overview, user characteristics
- **Functional Requirements** – system functions and features
- **Non-Functional Requirements** – performance, security, reliability
- **External Interface Requirements** – user, hardware, software interfaces
- **Constraints and Assumptions**

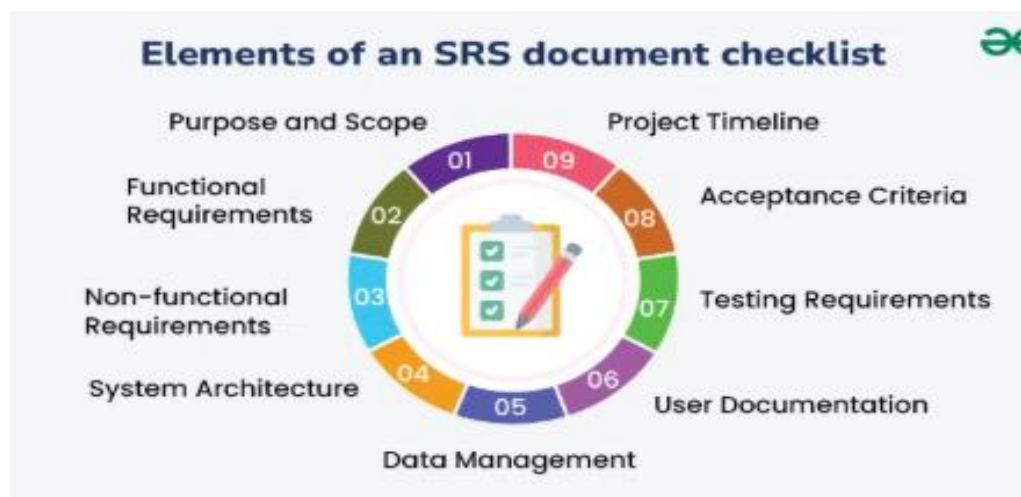
4. Validation and Review

- Review the SRS with users and stakeholders
- Ensure correctness, completeness, and consistency

5. Final Approval

- Obtain formal approval from the client
- Freeze requirements for development

SRS Preparation Diagram



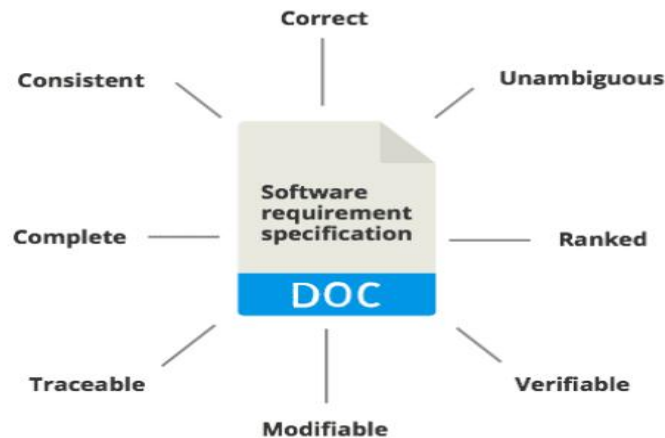
Possible Users of SRS

- Clients / Customers:** Understand system functionality and scope
- Project Managers:** Plan schedule, cost, and resources
- System Analysts:** Convert requirements into system models
- Designers & Developers:** Use SRS as a blueprint for design and coding

SE RTU Solution 2023

- e) **Testers / QA Team:** Create test cases and validation plans
- f) **Maintenance Team:** Understand system behavior for future updates

Uses / Importance of SRS



1. Acts as a **contract** between client and developer
2. Serves as a **basis for system design**
3. Helps in **cost and effort estimation**
4. Reduces misunderstandings and rework
5. Supports **testing and validation**
6. Assists in **system maintenance and enhancement**

Q5. Explain the following:

(a) Use Case Diagram

(b) State Chart Diagram

1. Use Case Diagram

A **Use Case Diagram** is a **UML behavioral diagram** that represents the **functional requirements of a system** by showing **interactions between users (actors) and the system**.

It answers the question:

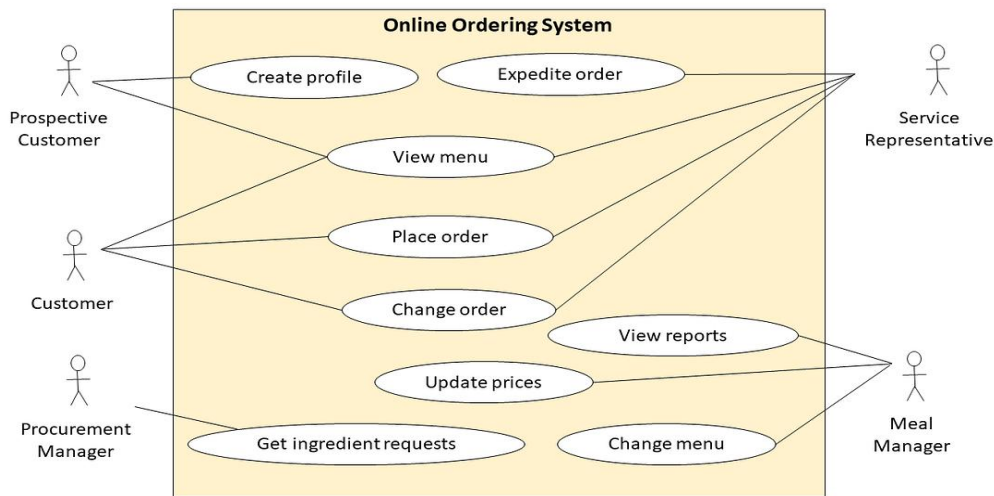
“What the system should do from the user’s perspective?”

Purpose of Use Case Diagram

- Captures **user requirements**
- Defines **system functionality**
- Provides a **high-level system view**

SE RTU Solution 2023

- Acts as a communication tool between users and developers



Main Components

1. Actor

- Represents a user or external system
- Can be a person, device, or another system

2. Use Case

- Represents a specific function or service
- Shown as an oval

3. System Boundary

- Defines the scope of the system
- Shown as a rectangle

4. Relationships

- Association
- Include
- Extend
- Generalization

Advantages

- Easy to understand

SE RTU Solution 2023

- User-centric
- Helps in requirement validation
- Useful in test case generation

Limitations

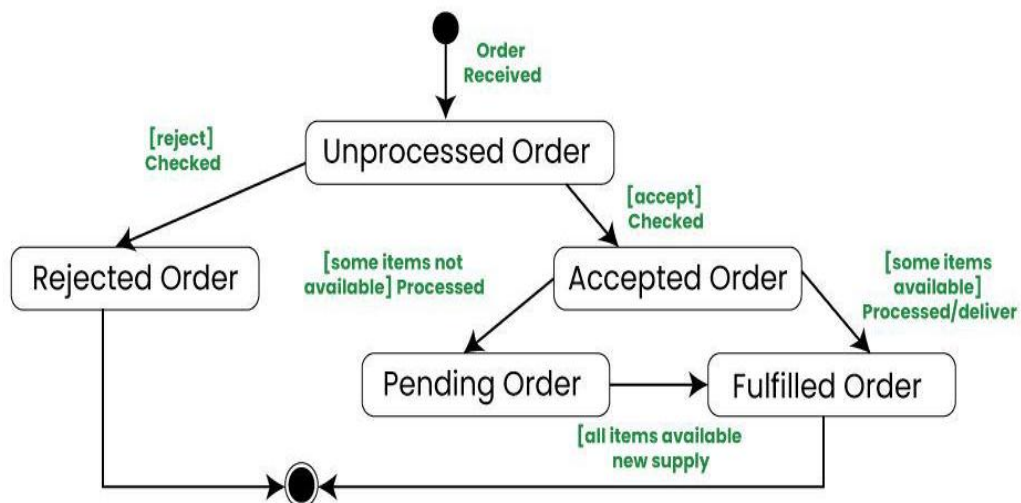
- Does not show internal logic
- No sequence or timing details

2. State Chart Diagram

A **State Chart Diagram** is a **UML behavioral diagram** that represents the **dynamic behavior of an object** by showing **different states and transitions** in response to events.

It answers the question:

“How does the object behave over time?”



Purpose of State Chart Diagram

- Models **object life cycle**
- Shows state changes due to events
- Useful for **real-time and embedded systems**

Main Components

1. State

- Condition in which an object exists
- Represented by a rounded rectangle

SE RTU Solution 2023

2. Event

- Trigger that causes state change

3. Transition

- Movement from one state to another
- Represented by arrows

4. Initial and Final State

- Initial state → solid circle
- Final state → double circle

Advantages

- Clearly shows object behavior
- Helps in system validation
- Useful for control systems

Limitations

- Complex for large systems
- Not suitable for data flow modeling